



ConvolQ — Full System Architecture

End-to-end pipeline: MediaSoup group/1:1 calls → FFmpeg audio extraction → Whisper transcription → Intelligence scoring → Leaderboard + Peer Matching. Privacy-first by design.

Version 1.0 Date April 2026 Status Draft Authors Engineering

∞ What This System Does

ConvolQ taps into live group or 1:1 voice calls via **MediaSoup SFU**, extracts per-speaker audio using **FFmpeg**, transcribes it with **Whisper**, scores each speaker's topic knowledge and language quality, and matches them with peers of similar skill — all without ever persisting their voice or transcript.



Privacy-first principle: Raw audio is processed in an isolated worker. The transcript exists in memory for ~3 seconds and is deleted immediately after embedding and scoring. Only irreversible numeric scores and encrypted vectors ever reach the database.

PIPELINE AT A GLANCE

- MediaSoup SFU — per-speaker RTP tracks**
Each participant in a group or 1:1 call has an independent audio Producer. Never mixed.
WebRTC · RTP **continuous**
- FFmpeg — RTP → PCM decoding**
Receives the RTP stream from MediaSoup, decodes the audio codec (Opus), outputs raw 16kHz PCM that Whisper consumes.
FFmpeg · Opus → PCM **continuous · 0ms lag**
- Whisper — transcription (memory only)**
Receives 15–30s PCM chunks with 2s overlap. Outputs text directly into the scoring pipeline. Never written to disk.
text: memory only **per chunk · 2–5s**
- Semantic chunking + topic detection**
Splits transcript at topic shift boundaries. Labels each chunk: finance, health, tech, etc.
BART-mnli · LangChain **per chunk · ~700ms**
- 3-dimensional intelligence scoring**
Topic depth (40%) + Factual accuracy (40%) + Language quality (20%) — per chunk, per topic. Factual accuracy runs async post-call.

LLM grader · LanguageTool **per chunk · 3-8s** **accuracy: async**

6 Embedding + raw text deletion

Convert text to a 384-dim vector. Then delete the text from memory. Vector is irreversible.

text: deleted ✗ vector: stored ✓

7 Qdrant + Postgres + Redis

Encrypted vectors → Qdrant. Numeric scores → Postgres. Leaderboard → Redis sorted sets.

AES-256 encrypted

8 Leaderboard + Peer Matching

Weekly/monthly/all-time leaderboards per topic. Peer matching by semantic similarity + score proximity. Both fire when the room ends.

Redis ZADD · Qdrant ANN **on room end · ~2s**

01 MediaSoup + FFmpeg Audio Ingestion

MediaSoup is a WebRTC SFU (Selective Forwarding Unit). It does not mix audio — it gives you a separate RTP track per speaker. FFmpeg then decodes each track into raw PCM audio that Whisper can process.

Why FFmpeg here?

MediaSoup outputs RTP packets carrying Opus-encoded audio. Whisper requires raw PCM at 16kHz mono. FFmpeg bridges this gap in a single command — it handles the RTP ingestion, Opus decoding, resampling, and pipe output all at once.

BASH

Copy

```
# FFmpeg: RTP stream → 16kHz mono PCM pipe to Whisper worker
ffmpeg \
  -protocol_whitelist file,rtp,udp \
  -i rtp://127.0.0.1:5004           # RTP stream from MediaSoup consumer
  -ar 16000 \                     # Resample to 16kHz (Whisper requirement)
  -ac 1 \                         # Mono channel
  -f s16le \                      # Raw signed 16-bit little-endian PCM
  -bufsize 512k \                 # Buffer for chunking
  pipe:1                          # Output to stdout → piped to Whisper worker
```

Per-speaker isolation

Each MediaSoup Producer (one per speaker) gets its own FFmpeg process

Each FFmpeg process pipes to its own Whisper worker instance

1:1 call = 2 FFmpeg processes. Group call with N speakers = N FFmpeg processes

All pipelines run in parallel — they never share state

Server-side wiring

JAVASCRIPT

Copy

```
const { spawn } = require('child_process');

// For each new MediaSoup producer (one per speaker)
router.on('newProducer', async (producer) => {
  const transport = await router.createPlainTransport({ ... });
  const consumer = await transport.consume({ producerId: producer.id });

  // Allocate a UDP port for this speaker's RTP stream
  const rtpPort = allocatePort();

  // Spawn FFmpeg: decode Opus RTP → 16kHz PCM pipe
  const ffmpeg = spawn('ffmpeg', [
    '-protocol_whitelist', 'file,rtp,udp',
    '-i', `rtp://127.0.0.1:${rtpPort}`,
    '-ar', '16000', '-ac', '1', '-f', 's16le', 'pipe:1'
  ]);

  // Pipe FFmpeg stdout directly to the transcription worker
  transcriptionWorker(producer.appData.userId, ffmpeg.stdout);
});
```



Key design decision: never mix audio tracks before transcription. If you mix, you lose speaker attribution and cannot score individuals. Always process one track per speaker.

02 Transcription Worker

The transcription worker receives the PCM pipe from FFmpeg, buffers it into chunks using Voice Activity Detection (VAD), and runs Whisper to produce text. The text is passed immediately to the scoring pipeline and never written to disk.

Chunking strategy for live audio

Buffer 15–30 seconds of PCM audio per speaker

Run Silero VAD to detect silence gaps (natural sentence boundaries)

Flush the buffer when silence is detected or the 30s limit is reached

Overlap 2 seconds between consecutive chunks to prevent cut-off sentences

PYTHON

Copy

```
import whisper, torch

model = whisper.load_model("base.en") # or "small" for better accuracy
```

```
def transcribe_chunk(pcm_bytes: bytes, speaker_id: str) -> str:
    # Convert raw PCM bytes to float32 numpy array
    audio = np.frombuffer(pcm_bytes, dtype=np.int16).astype(np.float32) / 32768.0

    result = model.transcribe(audio, fp16=False, language="en")
    text = result["text"].strip()

    # Pass directly to scorer – never write to disk
    score_and_embed(text, speaker_id)

    # Explicit deletion from memory
    del text, audio, result
```

Whisper model size trade-offs

MODEL	SPEED	ACCURACY	RAM	BEST FOR
tiny.en	~32× RT	Good	~390 MB	Low-power servers, real-time
base.en	~16× RT	Better	~740 MB	Good starting point
small.en	~6× RT	Great	~1.5 GB	Recommended for V1
medium.en	~2× RT	Excellent	~5 GB	High quality, GPU needed

03 Semantic Chunking & Topic Detection

A long conversation covers many topics. You need to know not just "how well does this person speak overall" but "how well do they speak about finance vs health vs technology." This requires splitting the transcript at topic shift boundaries — not by time or fixed token count.

How semantic chunking works

- Slide a 5-sentence window across the transcript
- Embed each window using a small, fast model (all-MiniLM-L6-v2)
- Compute cosine similarity between adjacent windows
- When similarity drops sharply (threshold ~0.65), a topic shift has occurred — split there
- Result: variable-length chunks, each about one coherent topic

Topic labelling

Feed each chunk to a zero-shot classifier or prompt an LLM directly:

PROMPT

Copy

Label the following conversation excerpt with exactly ONE topic from this list:
[finance, health, technology, education, sports, politics, science, general]

Return only the label, nothing else.

Text: "{chunk_text}"



For V1, use a fixed topic taxonomy (8–12 topics). In V2, switch to open-ended topic extraction to handle edge cases and niche subjects your users actually discuss.

04 Intelligence Scoring

Each chunk is scored across three independent dimensions. Every score is a number 0–100. No text is stored — only the numbers.



TOPIC DEPTH

40%

How deep is this person's knowledge of the topic? Looks for terminology, nuance, examples, and insight beyond surface level.

[LLM grader prompt](#)



FACTUAL ACCURACY

40%

Are the claims made actually correct? Uses RAG to pull relevant knowledge and cross-checks each assertion.

[RAG + LLM verifier](#)



LANGUAGE QUALITY

20%

Grammar correctness, fluency, vocabulary richness, sentence coherence. Modifier on top of content scores.

[LanguageTool + LLM](#)

Composite score formula

FORMULA

[Copy](#)

```
composite = (topic_depth × 0.40) + (factual_accuracy × 0.40) + (language × 0.20)

-- Example for a finance chunk:
-- depth=82, accuracy=79, language=74
composite = (82 × 0.4) + (79 × 0.4) + (74 × 0.2)
           = 32.8 + 31.6 + 14.8
           = 79.2
```

Score grader prompt (topic depth)

PROMPT

[Copy](#)

You are evaluating a person's knowledge depth on the topic of {topic}.

Score the following text on a scale of 0–100 where:

- 0–20 = Very surface level, no real knowledge shown
- 21–50 = Basic understanding, common knowledge only
- 51–75 = Solid understanding, correct use of terminology

76-90 = Deep knowledge, nuanced perspective, good examples
91-100 = Expert-level insight, original synthesis

Return only a JSON object: {"score": , "reason": ""}

Text: "{chunk_text}"

 **V1 recommendation:** Skip factual accuracy scoring initially — it requires a RAG pipeline and is expensive per chunk. Launch with topic depth + language only. Add factual accuracy in V2.

05 Privacy Layer & Embedding

After scoring, the text is deleted. What survives is a 384-dimensional float array (the embedding) and a few numbers (the scores). An embedding is mathematically irreversible — you cannot reconstruct words from it.

What is stored vs what is deleted

- ✔ **Stored:** Encrypted embedding vector (384 floats)
- ✔ **Stored:** Per-topic scores — depth, accuracy, language (numbers only)
- ✔ **Stored:** Topic label (e.g. "finance")
- ✔ **Stored:** Hashed user ID + timestamp
- 🗑 **Deleted immediately:** Raw PCM audio (never written to disk)
- 🗑 **Deleted immediately:** Transcript text (deleted from memory after embedding)
- 🗑 **Never stored:** Any personally identifiable content from the conversation

Embedding model options

OPTION	MODEL	WHERE IT RUNS	PRIVACY LEVEL
Best privacy	all-MiniLM-L6-v2 (ONNX)	On-device / browser via Transformers.js	Text never leaves device
Recommended V1	sentence-transformers (Python)	Isolated server worker	Text stays in your infra
High quality	text-embedding-3-small	OpenAI API	Text leaves your infra — DPA required



AES-256 encrypt every vector before writing to Qdrant. Store encryption keys in **AWS KMS** or **HashiCorp Vault** — never in the same database as the vectors. The server holding vectors should be unable to decrypt them without calling the key service.

06 Storage Architecture

Three stores, each with a distinct job. Never use one for another's purpose.

Qdrant

VECTOR STORE

- Per-speaker per-topic embeddings
- Named vectors: `finance_vec`, `health_vec`
- Payload: `user_id_hash`, `topic`, `scores`
- Filtered ANN search

Postgres

SCORE STORE

- Append-only `score_events` table
- `user_id`, `topic`, `composite`, `timestamp`
- Time-window leaderboard queries
- Consent & deletion audit log

Redis

LEADERBOARD CACHE

- Sorted sets per topic per window
- $O(\log n)$ rank lookup
- Refreshed hourly from Postgres
- Never source of truth

Postgres schema

SQL

Copy

```
CREATE TABLE score_events (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID          NOT NULL,           -- hashed, no PII  
  topic       TEXT          NOT NULL,           -- 'finance', 'health', etc.  
  depth       NUMERIC(5,2),  
  accuracy    NUMERIC(5,2),  
  language    NUMERIC(5,2),  
  composite    NUMERIC(5,2),  
  created_at  TIMESTAMPTZ  DEFAULT now()  
);  
  
CREATE INDEX ON score_events (user_id, topic, created_at DESC);  
CREATE INDEX ON score_events (topic, composite, created_at DESC); -- leaderboard queries
```

07 Leaderboard Engine

Weekly, monthly, and all-time leaderboards per topic. Always served from Redis cache — never a live Postgres query on the hot path.

Time windows — no data deletion, just query windows

This week — `created_at >= date_trunc('week', now())`

This month — `created_at >= date_trunc('month', now())`

All time — all rows, exponential decay weighting

Weekly resets do not delete data — the query window just shifts

Weighted recency SQL

SQL

Copy

```
SELECT user_id, topic,
       SUM(composite * CASE
           WHEN created_at > now() - interval '7 days' THEN 2.0
           WHEN created_at > now() - interval '30 days' THEN 1.5
           ELSE 1.0
       END) /
       SUM(CASE
           WHEN created_at > now() - interval '7 days' THEN 2.0
           WHEN created_at > now() - interval '30 days' THEN 1.5
           ELSE 1.0
       END) AS weighted_score
FROM score_events
GROUP BY user_id, topic
ORDER BY weighted_score DESC;
```

Redis sorted set pattern

REDIS

Copy

```
# Key pattern: leaderboard:{topic}:{window}
ZADD leaderboard:finance:week 94.2 user:alice
ZADD leaderboard:finance:week 89.1 user:bob

# Get top 10
ZREVRANGE leaderboard:finance:week 0 9 WITHSCORES

# Get a specific user's rank (0-indexed)
ZREVRANK leaderboard:finance:week user:alice → 0 (rank #1)

# Get user's score
ZSCORE leaderboard:finance:week user:alice → "94.2"
```

08 Peer Matching Engine

Find users who speak about the same topics *at the same skill level*. This is a filtered vector search — semantic similarity combined with score proximity in a single Qdrant query.

PYTHON

Copy

```

from qdrant_client.models import Filter, FieldCondition, MatchValue, Range

results = client.search(
    collection_name="speakers",
    query_vector=("finance_vec", current_user_finance_embedding),
    query_filter=Filter(
        must=[
            FieldCondition(key="topic", match=MatchValue(value="finance")),
            FieldCondition(key="depth_score", range=Range(gte=70, lte=90)),
            FieldCondition(key="active_30d", match=MatchValue(value=True)),
        ]
    ),
    score_threshold=0.75, # minimum cosine similarity
    limit=10
)

```

Match dimensions

Topic overlap — matched by which topics appear in both users' conversation history

Skill proximity — composite score within ± 15 points of the user's own score

Conversation style — cosine similarity ≥ 0.75 between topic embeddings

Recency — prefer peers active in the last 30 days

09 User Consent & Privacy Dashboard

Users must see exactly what happens to their voice before any recording starts. This is legally required under GDPR (EU), CCPA (California), and the DPDP Act (India).

Consent screen — shown before first analysis



What we tell users (plain language):

"We never store your voice recording."

"We never store the words you said."

"Your audio is processed on our server, turned into a skill score, and deleted — in under 5 seconds."

"We only keep your scores (numbers) and an anonymous style fingerprint of your conversation — not the words themselves."

"You can delete everything, anytime."

Privacy dashboard controls (must-have)

View all stored scores by topic and date

Delete scores from a specific conversation

Pause analysis — calls processed but nothing stored

Export full data as JSON (GDPR portability right)

Full account deletion — cascades to Qdrant + Postgres + Redis within 30 days



Under **GDPR Article 17**, users have the right to erasure. Deletion must cascade to all three stores. Log every deletion request with a timestamp for compliance audits.

10

Full Tech Stack

LAYER	COMPONENT	TOOL	WHY
Call infrastructure	SFU	MediaSoup	Per-speaker RTP tracks, no mixing
Audio decoding	RTP → PCM	FFmpeg	Opus decode, 16kHz resample, pipe to Whisper
Voice activity	Silence detection	Silero VAD	Natural chunk boundaries
Transcription	STT	Whisper / Whisper.cpp	Open source, self-hosted, private
Chunking	Semantic splitter	LangChain SemanticChunker	Topic-shift detection
Topic detection	Zero-shot classifier	BART-large-mnli	No training data needed
Depth scoring	LLM grader	Claude / GPT-4o	Nuanced knowledge evaluation
Language scoring	Grammar + fluency	LanguageTool + LLM	Open source + AI hybrid
Factual check (V2)	RAG verifier	LLM + knowledge base	Cross-check claims against facts
Embedding	Text → vector	all-MiniLM-L6-v2 (ONNX)	On-device capable, 384-dim
Vector store	ANN + filtering	Qdrant	Filtered vector search, named vectors
Score store	Event sourcing	Postgres	Append-only, time-window queries
Leaderboard cache	Sorted sets	Redis	O(log n) rank lookup
Encryption	At-rest encryption	AES-256 + AWS KMS	Keys never co-located with data

LAYER	COMPONENT	TOOL	WHY
Event queue	Stream decoupling	Redis Streams	Buffer between MediaSoup and workers
API	Backend	FastAPI (Python)	Async, connects all services

11 Data Flow — One Speaker's Audio

The complete journey from voice to leaderboard. The raw voice exists for approximately 3–5 seconds in memory and is never written to disk at any point.

- 1. Speaker audio → MediaSoup Producer → RTP packet stream (per speaker)
- 2. RTP packets → FFmpeg → Opus decode → 16kHz PCM pipe
- 3. PCM pipe → Silero VAD → 15–30s chunks with 2s overlap
- 4. Chunks → Whisper → raw text (**memory only — never written to disk**)
- 5. Text → Semantic Chunker → topic-labelled segments (memory only)
- 6. Segments → Scorer → {depth: 82, accuracy: 79, language: 74} per segment
- 7. Segments → Embedding model → vector[384] per segment
- 8. Raw text → **DELETED from memory**
- 9. vector[384] → AES-256 encrypted → written to Qdrant
- 10 Scores → appended to Postgres score_events table
- 11: Background job (hourly) → recomputes leaderboard → pushes to Redis
- 12. Peer match query → Qdrant filtered ANN → top 10 similar users returned

 The words the user spoke exist in memory for approximately **3–5 seconds**. They are never written to any persistent storage at any point in this pipeline.

12 Scaling Path

- ▶ MVP — 0 to 1,000 users
- ▶ Growth — 1,000 to 50,000 users
- ▶ Scale — 50,000+ users

13 Real-time vs Deferred Processing

The pipeline does not wait for the room to end. Processing runs in three distinct phases during and after a call — each with different latency characteristics and different triggers.



Short answer: FFmpeg and VAD run the entire call. Whisper + scoring fires every ~20 seconds per chunk. Leaderboard + peer matching fire once when the room ends. Factual accuracy runs async in the background after the room closes — it is too slow for the hot path.

The three processing phases

Phase 1 — Truly real-time continuous · 0ms lag

FFmpeg decodes the RTP stream and Silero VAD listens for silence gaps the entire time the call is live. This is just a pipe — no computation, no waiting. One FFmpeg process per speaker, running in parallel from the moment they join until they leave.

FFmpeg · Silero VAD

Phase 2 — Per-chunk processing every ~20s · 5-15s lag

Every time VAD detects a silence gap (or the 20s buffer fills), a chunk is flushed. Whisper transcribes it in 2-5s. Then semantic chunking, topic detection, and depth + language scoring run — another 3-8s. The partial score is written to Postgres and Qdrant immediately. The user's intelligence profile builds up incrementally throughout the call — not in a big burst at the end.

Whisper · Topic detect · Depth score · Language score

Phase 3 — Room close on room end · 2-4s

When the MediaSoup room closes (all peers disconnect), three things fire in sequence: chunk scores are aggregated into a final per-topic profile, the Redis leaderboard is updated via ZADD, and a peer matching query runs against Qdrant. Total time: 2-4 seconds. Users see their score card and peer matches within ~5 seconds of hanging up.

Aggregate · Redis ZADD · Qdrant ANN

Phase 4 — Async background post-call · 30-120s

Factual accuracy scoring cannot run in real-time — it requires a RAG pipeline that pulls relevant evidence from a knowledge base and asks an LLM to verify each claim. This takes 5-20 seconds per claim. It runs as a background job after the room ends, silently updating the accuracy scores once complete. The leaderboard reflects the updated score on the next hourly refresh.

RAG verifier · async job · leaderboard refresh

Timing reference table

STEP	WHEN IT FIRES	LATENCY	BLOCKING?
FFmpeg RTP decode	Continuous, whole call	0ms — just a pipe	No
Silero VAD	Continuous, whole call	~1ms per frame	No
Whisper transcription	Per chunk (~20s silence gap)	2-5s per chunk	Yes, per chunk
Semantic chunking	Per chunk, after Whisper	~200ms	Yes, per chunk
Topic detection	Per chunk, after chunking	~500ms (zero-shot)	Yes, per chunk

STEP	WHEN IT FIRES	LATENCY	BLOCKING?
Depth + language score	Per chunk, after topic	3–8s (LLM call)	Yes, per chunk
Write to Qdrant + Postgres	Per chunk, after scoring	~50ms	No
Aggregate final profile	Room end	~100ms (SQL)	No
Redis leaderboard update	Room end, after aggregation	~5ms (ZADD)	No
Peer matching query	Room end, after leaderboard	~200ms (Qdrant ANN)	No
Factual accuracy scoring	Async, post-call background job	30–120s total	No — background only



User-visible result: Score card + peer matches appear within **5–10 seconds** of the room closing. Factual accuracy score appears silently updated within 1–2 minutes. Users never wait for the whole pipeline to finish before seeing results.